

ne and aggregate codes

1. Elevate codes to themes, iteratively

1. Determine final themes

1. Assign final themes

1. Create a summary table

= repeat these steps until you have a set of themes that reflect the user experience

Detailed Notes on Steps in Verbatim Analysis

1. Identify your data by sorting and "cleaning" it.

Select the data that you're able to include in your analysis. We use the word "cleaning" to refer to the process of getting rid of the data that's not helpful to include in the analysis. Some of your data won't be appropriate to include in your analysis.

- Some verbatim will not be actionable because they're too vague and will need to be excluded from your analysis. For example, the response of "It's fine" doesn't tell us very much about the user experience and would be tagged as NA and put aside.

- Some of your verbatim will not be actionable because it isn't directly tied to your research goals or questions will need to be excluded from your analysis. For example, for the SUS verbatim analysis, we don't include verbatims that are only positive (e.g., 'GitLab is a fine tool to help developers') because they don't inform our research goal of understanding the current usability problems with GitLab and identifying improvements.

- Some of your verbatim will include a lot of different ideas or thoughts. In this case, you can clean your data by breaking those verbatim up so that they are only talking about one topic. This will allow you to more cleanly assign codes. For example, in the SUS analysis we broke up the verbatim paragraph in the table below into two separate verbatim, one about Navigation and one about Search. This makes it easier to assign codes for each section of the original verbatim.

Original Verbatim	Verbatim 1	Verbatim 2
-------------------	------------	------------

Original Verbatim	Verbatim 1	Verbatim 2
-------------------	------------	------------

"I think there are many things about GitLab, which I am not aware of especially the navigation, our team manages the tickets in GitLab but I was never able to find it in GitLab, I think search needs to be very much improved in GitLab providing more insights in the search bar or dashboard itself regarding everything."	"I think there are many things about GitLab, which I am not aware of especially the navigation, our team manages the tickets in GitLab but I was never able to find it in GitLab"	"I think search needs to be very much improved in GitLab providing more insights in the search bar or dashboard itself regarding everything."
--	---	---

2. Familiarize yourself with the data.

Read all of your data or a random sample of 25% of the data if you're working with a very large data set with more than 500 total responses - enough to get a sense of the verbatim that you'll encounter in the complete data set. Your goal is to get a general sense of what the data is like. You want to load the data in your mind to get a sense of it and so that you can confirm

that the final themes that you create reflect the data set.

3. Assign early drafts of codes.

To get started, create early drafts of the codes that you'll review later. This is the generative part of the analysis so you don't want to spend too long here. Instead, you can think of this as a short-hand note about what you saw in the verbatim. The goal is for you to be able to look over your list of early drafts of code so that you can see what codes can be aggregated.

In the tables below, we're using five example verbatims that all aligned with the same theme at the end of the analysis. Keep in mind that these verbatim wouldn't be grouped together like this during the early phases of the analysis.

Table with examples:

SUS Verbatim Early Draft of Codes	
----- -----	
I find the new "rules" section of gitlab-ci not intuitive. The former system was less complex and easier to understand.	CI rules not intuitive, complex
Pipelines are too complex and confusing	Pipelines confusing
My experience is that is really overly complex for my use case. I came over from bitbucket and this seems to just have way too much going on, when I just want to simply have a private repo that me and my small team work on.	
Lots going on, wants simplicity	
I find the way different pipelines are presented to be confusing and it's hard to know which are running	Pipeline presentation confusing
Any simplification wherever possible is welcome	Wants simplicity

Note: If you are performing your analysis in a spreadsheet, it's helpful to create a pivot table that aggregates all the codes that you've assigned so far so that you have a quick view of all of the early drafts of codes that you've assigned. It's best practice to do this with the complete data set of all the verbatims before you think about moving on to the next step of refining and aggregating codes.

4. Recursively refine and aggregate codes.

Revisit and refine your codes after you've looked at all of the verbatim, or at least another 25%, to see if you can refine them to make them more accurately reflect new verbatim and the data set overall. For example, you may start with a draft of a code like "feels lost" and then transition that to "confusing" to capture more verbatim with that code. We've included more examples in the table below.

During this part of the analysis you should ask yourself questions like:

- If I reframe this code in a different way, am I able to combine it?
- Are these two codes really that different?
- Am I combining codes so much that I'm not able to say something meaningful about each verbatim?
- Is there anything that I've read that isn't reflected in this list of codes - and if yes, what new code do I need to create to reflect that?

Note: If you are using a spreadsheet for your analysis, it's helpful to sort based on your early drafts of code (e.g., sorting them alphabetically) and to use the find function (e.g., search for all the times that the word complex is used within each code) to help you find ways to combine them.

Table with examples:

SUS Verbatim	Early Draft of Codes
I find the new "rules" section of gitlab-ci not intuitive. The former system was less complex and easier to understand.	CI rules not intuitive, complex
Pipelines are too complex and confusing	Pipelines confusing
My experience is that is really overly complex for my use case. I came over from bitbucket and this seems to just have way too much going on, when I just want to simply have a private repo that me and my small team work on.	Lots going on, wants simplicity
I find the way different pipelines are presented to be confusing and it's hard to know which are running	Pipeline presentation confusing
Any simplification wherever possible is welcome	wants simplicity

5. Elevate codes to themes, iteratively.

Now that you have an aggregated list of codes, look for the underlying themes that they inform and assign and refine themes. During this part of the analysis you're working to identify themes that help you to answer your research questions.

To do this, ask yourself questions like:

- What does this code tell us about the user experience?
- Does creating this theme allow me to combine codes?
- Does this theme address my research question or align with my research goals?
- If I create an aggregate theme does it become so broad that I lose a nuance of the data that I was seeing when I looked at the entire data set?
- If you're working in a spreadsheet, it's helpful to keep your original codes in a separate column from your themes so that you can iterate without losing work.

Table with examples:

SUS Verbatim	Early Draft of Codes	Next Iteration of Codes
I find the new "rules" section of gitlab-ci not intuitive. The former system was less complex and easier to understand.	CI rules not intuitive, complex	CI rules complex
Pipelines are too complex and confusing	Pipelines confusing	Pipelines confusing
My experience is that is really overly complex for my use case. I came over from bitbucket and this seems to just have way too much going on, when I just want to simply have a private repo that me and my small team work on.	Lots going on, wants simplicity	wants simplicity - private repo for team
I find the way different pipelines are presented to be confusing and it's hard to know which are running	Pipeline presentation confusing	Pipelines confusing

| Any simplification wherever possible is welcome | wants simplicity | wants simplicity |

6. Determine final themes.

Generally, you'll want to have a maximum of 10 themes. That's because the more themes that you have, the harder it will be for you to get a clear read on your data. Some themes will not be able to be combined, and that's ok.

You might need to take a step back here, both conceptually and in terms of an actual break to take a walk, in order to think about ways to reduce the number of themes. For example, in our SUS verbatim analysis we created a separate list of topics (e.g., Pipelines) for each verbatim (e.g., "Pipelines are too complex and confusing") so that we could create a shorter list of themes that reflected the user experience (e.g., theme: complex / confusing). This allowed us to reduce our SUS themes in verbatim analysis from a list of 23 themes to a smaller list of 10. Creating two different axes, or ways of looking at the data, is called a form of axial coding, which is a bit outside the scope of this handbook page. Suffice it to say that it can sometimes be helpful to create different types of themes to see how they relate to each other. As another example of using an additional coding axis, you might want to code each verbatim for valence, like the positive or negative emotions that they represent, so that you can see how user emotions expressed relate to different topics (e.g., which topics were talked about most positively).

Table with example:

SUS Verbatim	Final Code	Topic	SUS Theme
-----	-----	-----	-----
Pipelines are too complex and confusing		Pipelines confusing	Pipelines Complex/Confusing

7. Assign final themes.

Once you have your finalized list of themes, you'll need to go back through your data and make sure that you've assigned each verbatim with a theme. It's best practice to assign one theme to each verbatim because this allows you to report on the overall percentages of themes within your data set.

Table with examples:

SUS Verbatim	Final Code	Topic	SUS Theme
-----	-----	-----	-----
I find the new "rules" section of gitlab-ci not intuitive. The former system was less complex and easier to understand.		CI rules complex	CI /CD Complex/Confusing
Pipelines are too complex and confusing		Pipelines confusing	Pipelines Complex/Confusing
My experience is that is really overly complex for my use case. I came over from bitbucket and this seems to just have way too much going on, when I just want to simply have a private repo that me and my small team work on.		wants simplicity - private repo for team	Repository Complex/Confusing
I find the way different pipelines are presented to be confusing and it's hard to know			

which are running | Pipelines confusing | Pipelines | Complex/Confusing |
| Any simplification wherever possible is welcome | wants simplicity | Not
applicable | Complex/Confusing |

8. Create a summary table with examples for each theme.

After you define and assign your final themes (or during that process), create a table with examples of each theme as well as a quick description. This will make it easier for your stakeholders to understand what you've done.

Table with example:

SUS Verbatim	Quick Description	SUS Verbatim Example
-----	-----	-----
Complex/Confusing	User notes that there is a lot going on with GitLab and/or that it's very complex	"I think the way different pipelines are presented to be confusing and it's hard to know which are running"

You'll want to set aside some dedicated time to work on your analysis. Generally, your analysis will take 1-2 weeks to complete. It will take you a lot longer if you need to context-switch during your analysis because you'll have to spend time to refamiliarize yourself with the codes or themes that you've generated so far, and if you take too long of a break you'll have to review the data set so that you can make sure that your themes reflect what you saw in the entire data set.

SECTION: product/ux/ux-research/attending-a-research-event.md

Attending a UX research session can be enlightening! However, it's important to let the researcher take the lead on interactions with the participant. Some tips when attending sessions:

- If you have time, reach out to the researcher to let them know you'll be attending. Ask them how they prefer to receive questions during the session.
- Turn off your video.
- Mute yourself.
- Make sure to take notes in the appropriate Dovetail project.
- If you have questions you'd like the researcher to ask during the session, use their preferred method of communicating them.
- When the participant leaves the session, stay on video to share with the researcher what you learned from the session.

SECTION: product/ux/ux-research/choosing-a-research-methodology.md

When choosing the appropriate methodology (see handbook pages for problem validation and solution validation methods) for your research question, there are multiple factors to consider:

- The need to understand the details of an issue versus its frequency of occurring
- The characteristics of your users and their usage
- The fidelity of your product or designs

Each of these factors will dictate which methodology is most appropriate for your research questions. We'll go into each of these factors in more detail.

Detail versus frequency

Whether you are just starting out and wanting to get an idea of what problems may exist, the first factor to consider is if you already have an understanding of the problems users are facing. Based on the information you have about the problem, there may be a need to understand its severity or frequency relative to other known problems.

If you're in the initial discovery phase, it's important to be able to get into the weeds of your problem. You'll want the ability to ask detailed questions, follow up on participants' answers, and deviate from your discussion guide when interesting tangents present themselves. Thus, in-depth interviews are likely going to be preferable rather than a survey or other method where you have a fixed set of questions and no ability to follow up.

Once you have an idea of the types of issues your users are experiencing, you may want to understand how prevalent those issues are with the wider population. To do that, you need a suitably large sample of users so that you can be reasonably confident that your findings are representative of the user population as a whole. In this instance, a survey that you can distribute to potentially hundreds or thousands of users is a better fit. Using a survey with a fixed set of answer choices will allow for easy analysis of even a large number of responses.

Characteristics of your users

The next thing you'll want to think about when devising your study is the characteristics of your users' behavior or specific actions you're looking to study. Do all people perform a certain behavior in the same way, or does it vary, perhaps based on their role? Take for example administrators versus regular users. Their understanding and usage of the product will likely be very different. In that case, you would want to be sure you recruit people in different roles.

Does usage change over time? Perhaps you're interested in studying how people learn to use a new feature. It might be a good idea to talk to users at multiple intervals as they learn the feature, or at least recruit users at different points on the spectrum of usage.

The important thing is to think about how usage might be different for people based on who they are or how they're using your product and try to recruit people that reflect that different usage.

Fidelity of insight

People often ask at what stage of product development they should be conducting UX research. The reality is that you can conduct research at every stage of development with different levels of detail. The fidelity of research insights follows the fidelity of design.

If you have a hand drawn product flow on some pieces of paper, you can put that in front of users and get a basic, high level idea of how they comprehend the experience. The key phrase being high level, you won't get any detail and you shouldn't try. If you then turn that flow into a high fidelity prototype where users can click hotspots on a series of screen images to progress through the flow, you can get more detail, but still not as much as if they were interacting with a production interface.

The important thing to keep in mind is that you can do research at every stage of the development lifecycle, as long as you remain mindful of the limitations of what you can learn. However, even with these limitations, doing research at early stages is immensely valuable for providing product direction. Some information is so much better than no information.

SECTION: product/ux/ux-research/collecting-useful-data.md

Decide what kinds of insights to collect

As you take notes, it's easy to think that everything you hear and/or see is important, but this makes it extremely difficult to analyze the information across all your participants. You can use your research objectives to create categories you can reference as you take notes to make sure you are capturing the most salient information. Focus on the need-to-know information, not the nice-to-know information.

Decide on a collection method

Conducting user research often results in a pretty hefty amount of information. To stay organized throughout this process, spend some time upfront deciding how you'll collect your notes. Using Dovetail will help consolidate your data as well as facilitates highlighting and tagging content. Doing this makes it easier for your team to collaborate by adding their own notes and observations, and collecting everything in a central place will keep things organized and consistent.

Here are some note-taking resources to use, in addition to Dovetail, depending on the type of study you're conducting:

- A guide to user research note taking This guide shows how to take good notes to save time later on. It also compares alternatives to note taking in a spreadsheet -- for example, using sticky notes.
- User Interview Note Taking Template
- Usability Testing Rainbow Analysis Chart. This approach uses a templated and color-coded spreadsheet to record what participants did during the test. For a thorough walkthrough on how to use this method, check out this article.

Identify additional notetakers

A note-taking best practice is to have at least one note-taker for each research session. Having a note-taker helps by:

- Gaining insights and developing high-quality solutions, as observers interpret incidents

through their diverse professional points of view.

- Engaging the team constructively, as everyone understands usability problems together.
- Generating top findings together to get consensus quickly and motivate the team to fix problems.
- Multiple people often capture different insights

After scheduling your sessions with participants, promote the sessions within section or stage group Slack channels to solicit note-takers.

Using data from social media engagements

Twitter and LinkedIn are great platforms for engaging with our users and sometimes they can also prove to be a goldmine for user feedback. However, when using data received from these sources, we should proceed with caution and follow certain steps to avoid arriving at biased insights. It's important to ensure the feedback is not analyzed outside its original context.

- Determine the frequency of the feedback. How often are similar statements made? By how many people?
- Analyze the feedback to separate opinion from fact. Analyze the factual statements to see if solutions are proposed. If they are, determine what additional work is needed (either design or problem validation) to take the solutions to the next level.
- If a research project is currently underway and the feedback appears to be around the same topic, you can use it as an additional set of data to help triangulate the data from the research project.
- As a follow-up, request the person providing feedback to fill out the relevant issuable template, depending if it is a feature request or a bug, so they could easily create an issue around their feedback.

For more information around the guidelines for engagement on social media platforms, visit the Team Member Social Media Policy and Guidelines page.

SECTION: product/ux/ux-research/comparative-testing-for-navigation.md

Comparative, qualitative usability testing enables you to get feedback on 2-3 designs early in the design process to assess the pros and cons of different design directions. This is different from quantitative comparative studies that focus on benchmarking and measurement of the design.

The focus at this stage in the design process is to identify what is working well or not working well with different designs and provide insight on which one to move forward with. This is a within subjects methodology, meaning that each participant will see all designs. By experiencing multiple designs, participants are able to provide useful feedback because they are able to compare and contrast the different designs they've seen.

Why should I use this method?

If you have 2-3 navigation designs with distinct differences, and you want to see which one is resonating with participants before investing in further. You can use low fidelity prototypes

that have just enough interaction to test the tasks in the study.

Goal: Compare multiple early prototypes to see which performs better. If participants can navigate to a relevant task/tasks, assess overall usability, and obtain additional insight such as initial impressions, verbal qualitative feedback, etc.

Benefits:

- Participants can provide more beneficial and specific feedback because they're exposed to multiple designs. Having participants experience multiple designs to achieve the same goal will enable them to think about the differences between them.
- You will learn qualitative insights such as comments and opinions about the designs from participants thinking out loud.
- You will have quantitative data from Task Completion and UMUX Lite scores to compare your designs, although there are limitations to interpreting the data due to a small sample size. These should be interpreted as directional only.

What types of research questions can this method answer?

- Which navigation design performs better? Why?
- Which navigation design enables participants to complete their task/achieve most easily?
- Which proposed navigation layout makes the most sense? Why or why not?
- Do participants know where they are in the application? Why or why not?
- Are navigational UI elements standing out to participants? Why or why not?

Methodology details

Type of facilitation: This kind of testing may be moderated or unmoderated depending on the specific details and goals of the study (e.g., how much qualitative insight are you hoping to learn, can participants get through the prototype on their own, etc), but will likely be moderated to elicit enough qualitative feedback to compare the designs.

Fidelity required: A prototype with enough fidelity to test the basic interactions and be able to compare the differences between the designs. For example, if you want to assess whether participants can navigate to an area, you would need an interactive prototype that allows participants to complete that task, and ideally be able to click a couple of "wrong" areas or paths.

Recommended sample size: 5, because the goal is to obtain qualitative insights. If you are learning new and significant usability issues after 5 participants, add 1-2 additional participants.

Number of designs: It is recommended to limit to 3 designs to ensure participants have enough time to get through each design and reduce participant fatigue.

What to capture:

- Task completion: how many participants successfully completed each task?
 - how does this compare across the designs?
- UMUX Lite: UMUX Lite scores and reasoning for their ratings

- | Metric | Details |
|--|---|
| What it measures | |
| | |
| | |
| | |
| | |
| | |
| Task Completion | Assess if participants completed this task. This should be interpreted as directional only, given the small sample size. For example, this can indicate where to focus improvements/help to prioritize findings. |
| Effectiveness | |
| UMUX Lite | Two questions on a 7 point scale (1 = strongly disagree and 7 = strongly agree):
- 1. [This system's] capabilities meet my requirements.
- 2. [This system] is easy to use.
This should be interpreted as directional only, given the small sample size and the focus should be on why they provided the rating they did. Also collect qualitative feedback to add understanding to their answer by asking why participants rated their UMUX Lite as they did. |
| Perceived usability | |
| Usability severity rating | Usability issues can be rated as High, Medium, or Low.
- High: Participants could not complete a task or experienced significant frustration.
- Medium: Participants had some difficulties completing a task, but could complete it.
- Low: Participants experienced some minor frustration, confusion, or other issues. |
| How severe the issues are, informs priority to address | |

How to run a comparative usability test

- Test 2-3 designs max. Testing more than 3 designs may be overwhelming for participants and you may not have enough time in the sessions to cover them all with discussion.
- Test designs that have obvious differences to ensure that participants can distinguish them from each other.
- All participants should see all designs and be given the same tasks for each design, just ensure that you randomize the order of the designs to avoid order effects (tasks becoming easier with each design they experience due to learned behavior).
- Choose the most important tasks to test, mainly to keep the amount of tasks to a minimum. Because you are testing multiple designs in one session, you want to ensure you have time to get through everything.
 - For example, if you have 5 tasks and have 3 different designs, this is actually 15 tasks per participant.
 - Test the session length and gauge participant fatigue by running a pilot session first.

- Collect feedback from participants after each task, after each design, and at the end of the testing session:

- Post-task questions:
 - Do you have any feedback on the activity you just worked on?
 - Was anything particularly easy or difficult (if not already mentioned)?
 - Probe on specific things the participant was doing during the session (if moderated).
- At the end of each design:

(Give UMUX Lite survey first, before asking additional questions)

- Ask why they gave each the rating the design they did
- Do you have any feedback on the design you just saw?
- What did you like or dislike about this design?
- Probe on anything you observed during the test (if moderated).
- Can add more targeted questions depending on anything else you want to learn with the study.
- At the end of each test session, comparing all designs:
 - Allow participants to review all of the designs they saw first, to refresh their memory (if unmoderated, add a task at the end that allows participants to view all of the designs).
 - Considering the different designs you saw today, if you were to wake up tomorrow and see one of these designs integrated into your GitLab UI, which one would you like to see, and why?
 - What did they like? Why?
 - What did they dislike? Why?

How to report results

To report brief/initial findings in Slack or in an Issue, please use the following format:

- Number of usability issues found with each design
 - For example, "Design A had 5 usability issues, Design B had 2 usability issues, and Design C had 6 usability issues."
- Top 1-2 usability issues found in each design (highest severity) with a brief description of each
 - Fill out the usability severity analysis spreadsheet- (internal link) to determine severity of issues
 - For example, "2 out of 3 participants struggled to find a recent project with the new placement of the [button or element name]."
- Themes around behaviors that led to the usability issue
 - For example, "2 participants expected to find it [somewhere else]"
- Note any other interesting observations or feedback
 - For example, "All 3 participants expressed confusion around the change with [X] because."
- Note any issues that you plan to address before running the next 2 participants

SECTION: product/ux/ux-research/creating-first-click-study-qualtrics.md

These instructions allow you to create a first click study, using Qualtrics, to examine what a participant would click first in the interface to complete their intended task.

A combination of Text/ Graphic Question, Heatmap, and Timing question types are used to create task scenarios and record the respondent's first click on the image.

Here's how to set that up:

Step 1: After logging into Qualtrics, click on "Create a new project" that appears in the bottom left of the side panel that is called "Welcome to XM".

Step 2: Choose to create a blank survey project

Step 3: Add your task scenario using the Text / Graphic question type

You can skip Steps 3 through 6 below by importing the "Heat Map Question Block" from the Questions folder in the UX Research & Product Question Library

- Use a Text/ Graphic question type to create a place to describe your task scenario:
 - Create a new survey block
 - Select the +Add new question button that appears in the bottom right-hand corner of your new question block
 - When you've clicked into that question, you'll see the Question Type options displayed in the left panel
 - From the Question Type dropdown, select 'Text / Graphic'
 - Insert your task scenario in place of the text 'Click to write the question text'

Step 4: Add the graphic where you'd like your participants to perform their first click using the Heat Map question type

- Use the following steps to set up a Heatmap question:
 1. Select the +Add new question button that appears in the bottom right-hand corner of each question block
 1. From the +Add new question dropdown, select 'Heat Map'
 1. Click 'Select a graphic to use for this question'
 1. Select a graphic you've already uploaded to your Qualtrics account (or upload a new one from your computer)
 1. Set the number of clicks each respondent may make on the image ('1' is the default) in the left side panel
 - Note: If the respondent clicks more than the allowed number of times, their oldest click will be replaced with the newest one.
 - If you plan to analyze the data based on the user's initial instinct (one click only), you should allow one click.
 - If you want to evaluate multiple correct locations or see the user's first click and last click, you should allow more than one click.
 1. Click 'Add Region'
 - Adding regions will help you calculate the percentage of users who clicked in the correct location more efficiently.
 - If you do not add regions for the correct locations, all the clicks will be grouped as "Other" in the data report from Qualtrics.
 - Regions are not required for a Heat Map question, but they can make reporting easier.
 1. Click and drag the region to move it, or click and drag the corners to resize it.
 1. Click the text box beneath the region to type a region name (make sure to use a descriptive name as this name will not display to respondents, but will be used in your survey results).
 1. Create a region for each area of content your participants might select (e.g., a button or a sub-heading in the Navigation panel)

Step 5: Add a Timing question to track how long participants spend on the page with your first click question

- Select the +Add new question button that appears in the bottom right-hand corner of each question block
- From the +Add new question dropdown, select 'Timing'
- The timing question will not be displayed to participants, though it will show up if you preview the survey. That is done so that you can be sure the question is working.
- Insert a page break after the timing question by hovering underneath the question and clicking "+Add page break" when that option appears

Step 6: Add a question that will help you understand why users clicked where they did or how they felt about their first click

Knowing why users clicked where they did will help you understand how to adjust your designs.

- Add a Text Entry question to have the user explain their motivation for clicking in a certain area. Example: Why did you click there? (open text)
- You can also use a Multiple Choice question to have them rate their level of confidence.
 - Example: How confident are you that you clicked the correct location?
 - Completely confident
 - Somewhat confident
 - Neutral
 - Somewhat unsure
 - Completely unsure

SECTION: product/ux/ux-research/diary-studies.md

A diary study is a research methodology used to obtain feedback from participants over a period of time (days, weeks, or months). In these studies, participants are asked to document information on their own when they perform a specific action or the participant has reached a specific period in the research.

The purpose of doing diary studies

Diary studies can be very informative since we, as researchers, cannot observe everything a participant does within a given timeframe. A diary study helps by having participants keep track of information for us. The data from diary studies help researchers and teams learn about specific patterns or changes that emerge over time.

When to conduct diary studies

Diary studies are useful when there is interest in participants beyond a single interaction or touchpoint. This kind of research can examine participant feedback about a product (GitLab or other application), behavior (DevOps tool usage at work), or activity (submitting a merge request or searching for content online). Some examples of participant data that is common to collect in diary studies includes:

- Documenting what participants think when completing tasks

- Documenting the context in which participants complete tasks
- Documenting what participants do within a given timeframe
- Documenting pain points experienced while using the product over a month
- Documenting how participants learn using a new product within the first few weeks
- Documenting the number of times a participant performs an action or task (Note: Other research methods such as monitoring customer analytics data are more accurate in assessing the frequency of an action)

How diary studies are different from other types of research

Diary studies are unique compared to surveys, interviews, or focus groups because they gather data from the same group of participants, multiple times. This method overlaps more with longitudinal studies in situations where one or more quantitative measures (for example: confidence or effectiveness ratings) are collected at distinct points in time. The main distinction between the two methods is that diary studies do not require using quantitative metrics. Many diary studies can involve just collecting qualitative data (open ended feedback or self-reported observations).

How diary studies differ from longitudinal studies

Diary studies require many of the same steps outlined for longitudinal studies. One additional step is to determine when the participant will record entries:

1. On specific time intervals (for example: record once a day, two times a week, or every other week)
1. On a set schedule (for example: record only at day 10, day 20, and day 30)
1. While performing an action (for example: record as you interact with a feature in GitLab or use GitLab in a specific way)

When planning out the appropriate timeframes or touchpoints for the research, take into account the overall research questions.

- Example: If the research question is meant to understand how participants learn GitLab as a first time user, it would be better to have participants record entries once every few days to reflect on their experience at that point in time.

Other factors that can impact diary study research planning include the types of participants recruited, deadline for the research, and number of research questions.

How to set up a diary study

When setting up a diary study, it is important to think through all of the necessary parts participants will fill out within their diaries. Researchers should create a diary document that is easy to view, access, and edit. At GitLab, it is common to use collaborative digital files such as Google Docs or Google Sheets, so the researcher can share them with a participant and monitor their progress throughout the study.

When using Google Docs or Sheets, make sure to follow these steps and guidelines to ensure data privacy within your study:

1. Create a new Google Doc or Google Sheet
1. Add your instructions, tasks, and questions for the participant
 1. Make sure to provide a clear space or location within the file for participants to add their comments and feedback, so participants are not confused about what to do
1. In the initial meeting with a participant, go through the diary document in detail and answer any questions they have
1. Once the document is ready to share with a participant, click the share button at the top right hand corner
1. Keep general access defaulted to restricted
1. In the add people and groups field, add the email address of the participant
1. Give them editor access and type in a short note (if needed)
1. Select send

Depending on the number of expected diary entries and touchpoints, it is advised to open participants' diary documents often to monitor their current progress. If participants have not filled out specific entries on time, you can leave a comment and @ mention them using their email address to send a reminder to complete the task(s).

Tips for conducting diary studies

- Diary studies require providing a large amount of participant feedback, so it is crucial to find participants who are detailed in their diary entries and participants who are motivated to continue for the entire study.
 - Include an open ended question in your screener (for example: describe the ways you currently look for content on GitLab) or set up an initial interview with them to determine how verbose they are in their responses before inviting them to participate in the study.
 - Share study expectations with participants upfront, so they know what to expect from start to finish. By stating the overall research timeline to participants (number of touchpoints, number of expected diary entries), they will get a full understanding of what is required of them. This helps get buy-in from participants, so they are more motivated to see the study through until the end.
- Monitor and analyze participant diary entries as they are logged. This makes it easier to follow up with participants on any entries that are either unclear, lack sufficient detail, or do not correctly follow the study instructions.
- Participants can often forget to record their diary entries as they go through the study, so periodically send them reminders to complete the study tasks.
- Figure out the appropriate amount of touchpoints with participants in the study. In general, it's recommended to limit the number of total touchpoints to four at the most. If participants are starting to give the same kinds of responses after several diary entries, you may want to consider adjusting the number of overall touchpoints to avoid participants quitting the study early.
- During the study, set up short 10-15 minute interviews to speak with participants about their entries. Having a set time to discuss entries with the researcher makes participants feel more accountable. As a result, participants will usually put extra effort into writing out their diary entries.
- Attach small incentive amounts to each diary entry (see touchpoints 2 & 3 in the table below), so participants are more likely to stay in the study. Saving the largest incentive amount for the last diary entry can help the researcher meet desired sample sizes for the study and can avoid the need to recruit additional participants over time.

Touchpoint	Time scheduled	Gratuity amount	
-----	-----	-----	
1	45 minutes	\$30 USD	
2	15 minutes	\$20 USD	
3	15 minutes	\$20 USD	
4	60 minutes	\$200 USD	

Examples of diary studies

There are a few recent examples of diary studies completed by our GitLab team members:

- Global search diary study
- Navigation diary study (Sasha persona)
- Navigation diary study (Parker persona)

Additional resources on diary studies

These resources provide a more in-depth overview on how to conduct diary studies:

- Diary Studies: Understanding Long-Term User Behavior and Experiences
- User Interviews: Diary Studies
- How to Conduct a Diary Study: A Start-to-Finish Guide

SECTION: product/ux/ux-research/discussion-guide-user-interviews.md

How to write a discussion guide

{{< youtube "/sXE8F5Vu1sA" >}}

Video transcript

A discussion guide is a set of questions and topics that you would like to discuss with a participant during a user interview. It typically consists of an introduction, warm-up questions, exploratory questions and a debrief. Today, I'm going to walk you through how to create a discussion guide.

Introduction

Introduce yourself and let the participant know what to expect during the interview. Give them a chance to ask questions. Most people have not been interviewed before, so take some time to put them at ease. Prior to the interview, you should have already obtained written consent to record (through a screener question or email communication). However, it's a good idea to double check verbally that the participant agrees to be recorded and consents to having the recording shared internally at GitLab.

Warm-up questions

Start by asking the participant a few easy questions about themselves and their job. This will

help the participant get used to the process of answering questions. It's also an opportunity to begin building rapport with the participant, so that they are more inclined to open up to you when you begin asking exploratory questions. Listen closely, their answers may help provide context for any later responses they give. Some warm-up questions you could ask are:

- On the screening survey, you mentioned that you are a DevOps Engineer. How long have you been in that role?
- What kind of work does your company do?
- What does your typical working day involve?
- What tasks do you commonly do within GitLab?

Exploratory questions

When you start writing your exploratory questions, you'll want to group questions into common topics, so that your interview naturally flows from one topic to the next. As you begin to structure your questions, allocate time for each topic. This will help keep your interview on track. Move from general questions to more specific questions related to your research goals. For example 'How do you currently go about this task?' to 'What's the hardest part about task?' to 'What could be better about how you currently do this?'. At the same time, don't leave your most important questions until the very end in case a user spends more time than you anticipate answering an earlier question. By allocating time to each topic ahead of time, you can assess when you've spent too much time on one topic and need to skip ahead to another topic.

It's okay to ask questions about past experiences, as long as you recognize the limitations of people's memory. The human memory is fallible and it can often be difficult for people to remember specific details. For example, if I asked you whether you had breakfast three days ago? You could probably tell me 'Yes' or 'No'. Yet, if I asked you to recall how long your breakfast took to eat, you'd probably struggle to provide an answer or you might even be tempted to hazard a guess. Ask questions which delve into participants' general experiences and opinions but don't press participants for details they can't provide. Otherwise, they may be tempted to make up their answers.

Participants can't predict the future. If you ask them a question like: 'Would you use this feature?' their response may not be an accurate reflection of what they would actually do. For example, some people might say 'No' because they might not be able to visualise how the feature would work from a description alone. Others might say 'Yes' because they don't want to rule out the possibility that at some point in the future the feature might be useful to them.

Instead of asking people to think about how they might act in the future, ask them about what they are doing in the present and/or what they have done in the past. Instead of asking "Would you use this feature?" ask instead "How are you currently accomplishing this task?" or "How have you accomplished this task in the past?" However, if you are asking about past experience, be aware that there are limitations on memory. As a best practice, don't ask about past behavior that occurred more than 2 months ago. In those cases, it's best to stick to asking about the present.

If you're interested in improving your interviewing skills, consider signing up for the Interview Carousel, a lightweight interview training program run by the UX Research team.

Debrief

Thank the participant for their time and explain what happens next with the feedback they have given you today. Give the participant a chance to ask any questions about the research. If you are paying a participant for taking part in your study, ensure you share details of how they will be paid and when they can expect payment. Leave your contact details with them in case they have any follow-up thoughts they want to share with you.

After your guide is written

Once you have written your discussion guide, you should rehearse and test out your guide with a colleague. This will give you a sense of how long your script will take to run through, and it will help you spot any questions that people may have difficulty answering.

Discussion guide and interview tips

- Remember your discussion guide is just that: a guide. It's a reference tool to help facilitate conversation.
- If a participant says something interesting that is not covered by your guide, listen to them and explore what they are saying. You may uncover something you hadn't previously considered.
- Active listening is key. You should react to what your participant is saying.
- Ask open ended questions to keep a conversation flowing.
- Remember to probe participant's answers. Ask: Why? Why not? Who? What? Where? When? How?
- Never ask participants what they want.

Example discussion guide

- Discussion guide template

Example questions for user interviews

Note: Don't use all the questions below! Select a couple to build your discussion guide.

Warm-up questions

- What's your current role?
- What made you become a [job title]?
- How long have you held the role of [job title]?
- What kind of work does your company do?
- What does your typical working day involve?
- What would you say are the top 3 tasks in your role?
- How long have you used GitLab?
- What do you use GitLab for?

Exploratory questions

- What are you trying to [achieve/get done]? Why?
- How do you currently go about this [task/process]?
- Can you show me how you currently do this [task/process]?
- Has anything about the way you do this [task/process] changed over time? How? Why?
- Do you anticipate [task/process] changing? Why?
- What do you like about [task/process]?

- What's the hardest part about [task/process]?
- What could be better about how you currently do this [task/process]?
- Tell me about the last time you tried to [task/process]
- Why do you keep doing [task/process] Why is it important to you?
- How often do you do [task/process]?
- How often do you experience [problem]?
- What happens before/after you experience [problem]?
- What work-arounds have you implemented to help you with this?
- What are you currently doing to make this [task/process] easier?
- Are you looking for an alternative solution for [task/process]? Why? Why not?
- How does this [task/process] impact other areas of your work?
- Could you describe step by step how you do [task/process]?
- What step is the most time-consuming? Why is that?
- If you could remove one step from the process, which step would it be?
- What other products have you tried?
- How did you hear about these other products?
- Why did you decide to try [product]?
- What made you choose [product] in the end?
- What do you like about [product]?
- What do you dislike about [product]?
- What's the hardest part about using [product]?
- How does the cost of [product] compare to the value it provides?
- How important is [product] to your organization? How would you rate its level of important on a scale of 1 to 10, with 10 being most important?

SECTION: product/ux/ux-research/finding-existing-research.md

Before starting new UX research, a good practice is to first look to see if your research questions can be answered through research that was already conducted. This step doesn't take long and can sometimes result in saving a significant amount of time if you're able to find the answers to your questions.

How to find existing research

- Dovetail is the single source of truth (SSOT) for all user insights discovered by GitLab Team Members.
- UXRI Insights project - Prior to Dovetail, GitLab was a research repository. Instead of reports and slide decks, we used issues to document key findings from research studies. A directory of completed research is available in the project's ReadMe file. We're currently working on an integration to surface up insights from Dovetail into the UXRI Insights project. In the meantime, feel free to explore both Dovetail and the UXRI Insights projects to look for insights.
- GitLab Community Forum is run by community members. They share, ask, and discuss everything related to GitLab.
- Zendesk is a ticketing system used by GitLab's Support team to track problems raised by customers. Zendesk accounts are available for all GitLab staff.
- Chorus.ai is used by our Sales team to record conversations that they have with GitLab customers. It has the ability to transcribe and highlight key points during these conversations. Open an access request.

- GitLab issue tracker

SECTION: product/ux/ux-research/finding-saas-users.md

Finding SaaS (GitLab.com) users

When to look for users in the data warehouse

We have a variety of methods for identifying and recruiting participants for research studies, including Respondent.io and recruiting via social media. However, there are situations where these methods aren't suitable and you need to be able to find users based on a very specific criteria backed by objective usage data rather than self-reported or inferred data. The best way to do this is to take advantage of the usage and demographic data we have in our data warehouse.

A non-exhaustive list of examples of when finding users via the data warehouse would be appropriate:

- You are looking for users with a certain volume of usage of a feature or stage rather than simply being users of a stage
- You are looking for users that have interacted with a specific type of object or content on GitLab (ex: epics, snippets, wiki, etc.)
- You need to understand the broader context of the group or project the user is working within (ex: users in groups with less than 30 users, users in Gold plan groups, users in projects with no repository usage)

Pre-requisites

1. You query the data warehouse using Structured Query Language (SQL). There are countless guides to learning SQL available for free on the Web. There are many different flavors of SQL, and the one we use is Snowflake. However, most basic functionality is consistent among the major variations of SQL, so don't feel like you need to seek out Snowflake-specific resources.

1. In order to query the data warehouse, you'll need Editor access to Sisense (formerly Periscope). This requires completing an access request.

1. Data for Product Managers provides an overview of how to gain access to the SQL Explorer in Sisense where you will write and execute your queries, as well as providing a high level overview of how the warehouse is structured.

1. The Data team documents individual data warehouse tables using a tool called dbt. Some tables are better documented and contain purpose descriptions and explanations of different columns, and some unfortunately do not. However, documentation does improve over time.

SaaS-specific tables

Our data warehouse contains data for a variety of different areas. For our purposes, we're interested in the data detailing user information and usage of GitLab.com (SaaS).

Most of the tables housing data related to SaaS use the prefix `gitlabdotcom`, though not all. A good starting point would be to explore the different tables with this prefix and look at what

data they contain.

Users Table

The our overall users table for SaaS is gitlabdotcomusersxf. It contains the most complete picture of individual users, including information such as account creation date, days active, role, and highest paid plan (if any). This should be your starting point and your source of truth. the userid variable found in this table is our primary identifier for GitLab.com users, and is used in other tables to identify the user with which a record is associated. For example, the gitlabdotcommergerequestsf table houses records of MRs created on GitLab.com, and contains a column called authorid to identify the author of a given MR. This authorid is a GitLab.com user ID, and you'd be able to connect the author of that merge request to the record in the users table (or any other table) using that userid variable.

You can use this table to find GitLab.com user IDs for 100 users that created their account in the last seven days:

```
sql
SELECT userid FROM analytics.gitlabdotcomusersxf
WHERE accountage >= 7
LIMIT 100
```

Object Tables (projects, issues, merge requests, etc)

We have a tables containing records for all of our "top level" objects in GitLab, among others:

- projects
- groups
- issues
- merge requests
- epics

If you can create it on GitLab.com, there is probably a table for it. These tables follow the same structure as other SaaS tables. For example:

- gitlabdotcomissuesxf
- gitlabdotcomepicsxf
- gitlabdotcommergerequestsf

The records in these tables are not full fidelity. They may not include all data shown when the object is rendered in the interface. When you load an issue in the GitLab.com UI, you see comments and all sorts of metadata. Much of this data will not be found in the issues table in the warehouse, and is either not accessible or stored in separate tables to keep tables to a reasonable size. Additionally, we do not expose data in the warehouse that would otherwise be private. For example, while you can see titles and descriptions for issues that are public on GitLab.com (just as you would if you viewed them on the Web), you will not be able to see that information for non-public issues.

These tables are good for when you want to understand how many of something a user or a group

has created or interacted with, or if you want to start with a certain type of entity (say, an issue that was promoted to an epic) and then see what users have interacted with such an entity. These tables do not provide event level data, so you will not necessarily be able to tell when or exactly how a user created or interacted with something, just that they did (or did not).

For example: You're researching issue weights and you want to find the user IDs for 100 users who have created an issue with a weight assigned in the last 30 days. You also need to know how many of those issues they've created over that period. You would use this query:

```
sql
SELECT
  authorid,
  COUNT(1) as numissues
FROM analytics.gitlabdotcomissuesxf
WHERE
  weight IS NOT NULL
  AND issuecreatedat >= DATEADD(day, -30, currenttimestamp)
GROUP BY 1
LIMIT 100
```

Stage Monthly Active Users

We don't log user IDs on specific event records, so you can't find out exactly when a certain user completed a specific action. However, we do log how many times per month and days per month a user interacts with different high level features and stages in the gitlabdotcommonthlystageactiveusers table. This is good for criteria where you're looking for "light" or "heavy" users of certain features or stages, or when you want to make sure you get users with at least some minimum recent usage of a stage.

For example: You need to find user IDs for 50 users who interacted with CI pipelines at least 17 times OR on 9 different days in a certain month (or more accurately, the last 28 days of a certain month):

```
sql
SELECT userid
FROM analytics.gitlabdotcommonthlystageactiveusers
WHERE
  eventname='cipipelines'
  AND (eventcount >= 17 OR eventdaycount >= 9)
  AND smaumonth='2020-04-01'
LIMIT 50
```

How to get help

- For questions on where you can find certain data in the warehouse, the data channel on Slack can address them.
- If you require a very in depth SQL query, you can reach out to the Product Data Insights team

to see if they have availability to assist you.

SECTION: product/ux/ux-research/first-click-testing.md

What is first click testing?

First click testing is used to determine if participants are able to find something they're looking for in an interface within their first click. Assessing the first click is important because it is much more likely that a user will succeed in completing a task if they're able to click on the right path initially. First click testing will be able to tell you if participants are clicking the intended area for a given scenario. If they don't, you can learn where else they would click, and why.

Why should I use this method?

You have a navigation design that's in the very early stages (it may just be a static mock or even a sketch) and you want to know if participants are able to correctly identify where to go to find certain elements/information, and if not, identify where they are going instead.

Goal: Learn where participants would initially click to complete a task and why. This testing is earlier than testing broader usability and will help to validate design ideas for navigation changes.

What types of research questions can this method answer?

- Do participants know where to first click to get to X?
- Where do participants click for a given task? Why?
- Does the proposed layout make sense?
- Does the hierarchy of your information architecture make sense?
- Do participants know where they are in the application?
- Are UI elements standing out to participants?

Methodology details

Type of Facilitation: Unmoderated

Fidelity required: static images, even a sketch if there is enough detail

Recommended sample size: 30 participants. There are additional factors that impact the sample size indicated for each study, please work with your UXR to determine this.

Data to collect:

- Quantitative: The table below outlines two quantitative metrics to capture in a first click test. This data should be explained further with qualitative insights to understand why participants made the selections they did. The goal is to understand their behavior to learn why the design is performing the way it is.

|Metric|Details|What it measures|

|----|----|----|

|Findability rate |If there are multiple correct options, a breakdown of how often each choice was selected (%)|Task success (can participants find the correct item?)|

|Confidence |1 = Not at all confident, 2 = Not very confident, 3 = Somewhat confident 4 = Very confident, 5 = Extremely confident)
 And why they chose that rating| How confident participants are with their choice|

- Qualitative: Collect qualitative feedback to add understanding to their answers:
 - Ask why participants made their selection (open-ended question): Why did you click there?
 - Ask for their expectation for the selection: What would you expect to see after clicking there?

Recommended testing platform: Qualtrics

How to set up a first-click test for navigation

- First, determine which tasks that you want your participants to complete. Choose tasks relevant to what you're testing from the Core Navigation tasks list (in process).
- It's recommended to have five tasks or less for the first click test.
- Randomize the order of tasks if you have more than one (unless tasks are dependent on each other and only make sense in a certain order). You can set randomization up easily within Qualtrics.
- A Navigation First Click Testing template is located in Qualtrics, within the UX Research & Product library (internal link). For more details on how this template was set up or how to use Qualtrics for a first click study, please refer to the Creating design evaluations in Qualtrics handbook page.

How to report results

To report brief/initial findings in Slack or in an Issue, please use the following format and utilize this template to create a chart:

- % of participants (A out of BB) correctly clicked the first item (note which item/items would be counted as correct).
- For the correct selections, the average confidence level was X/Z
- Note any themes about their correct choice "6 participants mentioned that they would expect to find X in choice B because."
- Other common incorrect first click choices were:
 - Choice B: 27% of participants (6 out of 20)
 - Choice C: 27% of participants (6 out of 20)
- If there is anything to note about their confidence level and incorrect choices:
Interestingly, participants felt more confident on average in choice B (3.5) than Choice A (2)
- Note any themes about their incorrect choice "4 participants mentioned that they would expect to find X in choice B because."
- Can note any other interesting finding or verbatim.

If there's any context that you can provide in your interpretation of the results, that will be helpful. For example, if participants selected Merge Requests in the left sidebar, you can add that the common incorrect first click in this area may be due to the task calling out a merge

request.

SECTION: product/ux/ux-research/interview-carousel.md

At GitLab, everyone can contribute, which applies to UX Research, too. Product Managers and Product Designers regularly speak with users to validate the problem spaces and solutions they are working on